

// JavaJub

Собес в Лигу Цифровой Экономики Java Middle

Вопросы, задачи и подготовка
к live-coding и интервью

Стек

Java 11+ • Spring • Hibernate • PostgreSQL • JUnit • Docker • Jenkins

Содержание

01	Про Лигу и формат собеса	3
02	Стек по вакансии	4
03	Java Core — что точно спросят	5
04	Коллекции	7
05	Многопоточность и JMM	8
06	Spring и Spring Boot	10
07	Hibernate и Spring Data JPA	12
08	SQL и PostgreSQL	13
09	Тесты: JUnit, Mockito, TDD	14
10	Docker, Jenkins, Linux	15
11	Практические задачи (live-coding)	16
12	План подготовки + чек-лист	21

01 Про Лигу и формат собеса

Лига Цифровой Экономики — крупный российский IT-интегратор (6 000+ человек, больше 1 000 проектов в год). Клиенты: Сбер, ВТБ, Альфа-Банк, Ростелеком, МТС, Роснефть, МВД, министерства. Для Java Middle это значит: проекты чаще всего банковские или государственные, с жёсткими требованиями к качеству и тестированию.

► Как устроен процесс

Этап	Длительность	Что проверяют
Скрининг HR	20–30 мин	Мотивация, ЗП-ожидания, согласие на ТК РФ
Тех-интервью	60–90 мин	Java Core + live-coding + SQL
Интервью с руководителем	30–60 мин	Проект, команда, матч по софтам
Проверка СБ	до недели	Стандартная для госпроектов
Оффер	—	По ТК РФ, ДМС, стоматология

► Как проходит техническая секция

- Видео-созвон с демонстрацией экрана + совместный редактор кода.
- Обычно одним днём — собес часто занимает одну встречу на ~1 час.
- Меньше теоретической зубрёжки, больше практики и рассуждений.
- Обязательно будет живое кодирование и вопросы по SQL.
- Могут попросить разобрать кусок чужого кода или объяснить баг.

ФИШКА · Ключевая особенность Лиги

По отзывам кандидатов на DreamJob: «Лайфкодинг, понимание SQL. Спрашивают твоё умение мыслить, а не теоретическую подготовку». Готовься не пересказывать учебник, а писать код и объяснять, почему именно так.

02 Стек по вакансии

На апрель 2026 года на Хабр Карьере актуальна вакансия Java Middle в Лиге — в команду антифрод-решения ведущего банка. Стек оттуда — наш основной ориентир.

► Обязательный минимум

- Java 11+, опыт от 2 лет
- Spring 5–6, Spring Boot 2–3
- Hibernate, Spring Data
- PostgreSQL (реляционные БД)
- Maven
- JUnit 5, Mockito
- Hamcrest, AssertJ
- Docker, Openshift
- Linux (уверенные знания)
- Jenkins
- TDD и умение писать unit-тесты

► Будет плюсом

- Опыт разработки на Kotlin
- Микросервисная архитектура
- Опыт code review
- Опыт работы в банковских проектах

ВНИМАНИЕ · Что это значит для подготовки

Стек приземлённый — никаких Virtual Threads, ZGC, GraalVM и прочего хайпа. Ждут крепкое владение Java 11 + Spring + Hibernate + SQL + тестами. Всё остальное — бонус.

03 Java Core — что точно спросят

► Базовые вопросы

- Что такое JDK, JRE, JVM?
→ JVM — виртуальная машина (исполняет байт-код). JRE = JVM + стандартные библиотеки. JDK = JRE + инструменты разработки (javac, jdb, jar).
- Области памяти JVM.
→ Heap (объекты), Stack (фреймы методов, примитивы, ссылки), Metaspace (метаданные классов), PC Register, Native Method Stack.
- Почему String immutable?
→ Безопасность (передача в файлы, БД, сеть), потокобезопасность, кэширование hashCode, работа String Pool.
- Что такое String Pool?
→ Область в heap, где хранятся уникальные строковые литералы. String s = "hi" кладёт в пул, new String("hi") — создаёт новый объект в heap.
- Разница между == и equals() для строк.
→ == сравнивает ссылки, equals() — содержимое. Для литералов == может дать true из-за String Pool, но полагаться на это нельзя.
- Что вернёт someObj.equals(null)?
→ false. По контракту equals не должен бросать NPE на null-аргумент.
- Контракт equals/hashCode.
→ Если a.equals(b), то a.hashCode() == b.hashCode(). Обратное не обязательно. Переопределяешь один — переопределяй и второй.
- Что сломается, если hashCode() вернуть константу?
→ Все объекты попадут в один bucket HashMap. В Java 8+ при 8 коллизиях список превращается в red-black tree, но O(log n) вместо O(1) — всё равно деградация производительности.
- Разница между checked и unchecked исключениями.
→ Checked наследуются от Exception (но не RuntimeException) — компилятор требует throws или catch. Unchecked — от RuntimeException/Error — обрабатывать необязательно.
- try-with-resources — какой интерфейс нужен?
→ AutoCloseable (или Closeable). Ресурсы закрываются автоматически в обратном порядке объявления.
- Что такое функциональный интерфейс? Примеры.
→ Интерфейс с ровно одним абстрактным методом. @FunctionalInterface — опциональная аннотация. Примеры: Function, Predicate, Consumer, Supplier, Runnable, Comparator.
- Stream API: промежуточные vs терминальные операции.
→ Промежуточные (filter, map, sorted) возвращают Stream и ленивые. Терминальные (collect, forEach, count, reduce) запускают pipeline.
- Разница между map и flatMap.
→ map: T → R. flatMap: T → Stream с уплощением результата.
- Когда использовать Optional?
→ Для возвращаемых значений методов, где результат может отсутствовать. НЕ для полей, параметров методов, коллекций — это антипаттерн.
- Можно ли переопределить static-метод?

→ Нем. *Static*-методы не участвуют в полиморфизме. В подклассе это будет сокрытие (*hiding*), а не переопределение.

- **final** у класса, метода и поля.

→ Класс — нельзя наследовать. Метод — нельзя переопределить. Поле — нельзя переписывать (но можно менять внутреннее состояние, если поле — ссылка на *mutable*-объект).

- Абстрактный класс vs интерфейс.

→ Абстрактный класс может иметь состояние и реализацию, наследуется только один. Интерфейс — контракт, реализуется любое число. С *Java 8* в интерфейсах появились *default* и *static* методы, с *Java 9* — *private*.

СОВЕТ · Лайфхак

На вопрос про `equals/hashCode` обязательно приведи пример: «если положить объект в `HashSet` и потом изменить поле, которое участвует в `hashCode` — объект потеряется, `contains()` вернёт `false`». Это показывает понимание, а не заученность.

04 Коллекции

HashMap — чемпион по частоте вопросов. Готовься рассказывать «как устроено».

- Основные интерфейсы Collection Framework.
→ Collection (List, Set, Queue), Map (отдельно, не Collection). List — упорядоченный с дубликатами. Set — без дубликатов. Map — пары ключ-значение.
- Как устроен ArrayList?
→ Динамический массив. При нехватке места создаёт новый массив в 1.5 раза больше и копирует через Arrays.copyOf.
- ArrayList vs LinkedList.
→ ArrayList: $O(1)$ доступ по индексу, быстрая итерация, локальность в кэше. LinkedList: $O(1)$ вставка в начало, но $O(n)$ поиск. На практике почти всегда выигрывает ArrayList.
- Как устроен HashMap?
→ Массив бакетов (Node[]). Индекс бакета: $(n - 1) \& \text{hash}(\text{key})$, где n — размер массива. Коллизии — связный список. С Java 8: при 8 элементах в бакете и размере таблицы ≥ 64 список превращается в red-black tree.
- Что такое load factor?
→ Порог заполнения, по умолчанию 0.75. При $\text{size} \geq \text{capacity} * \text{loadFactor}$ — resize: новый массив вдвое больше + перехэширование всех элементов.
- Сложность операций HashMap.
→ put, get, remove — $O(1)$ в среднем. В худшем случае $O(\log n)$ благодаря дереву (Java 8+).
- Можно ли null-ключ в HashMap?
→ Да, один null-ключ, кладётся в bucket 0. В ConcurrentHashMap нельзя ни ключ, ни значение.
- HashMap vs ConcurrentHashMap.
→ HashMap не потокобезопасен. ConcurrentHashMap потокобезопасен: в Java 7 — Segment-блокировки, в Java 8+ — CAS + synchronized на головах бакетов.
- HashMap vs TreeMap vs LinkedHashMap.
→ HashMap — хэш, без порядка, $O(1)$. TreeMap — красно-чёрное дерево, отсортирован по ключам, $O(\log n)$. LinkedHashMap — сохраняет порядок вставки или access order.
- Fail-fast итератор.
→ Кидает ConcurrentModificationException при изменении коллекции не через сам итератор. Так работают итераторы HashMap, ArrayList.
- Как сделать List неизменяемым?
→ List.copyOf(list) (Java 10+) или Collections.unmodifiableList(list). На add/remove — UnsupportedOperationException.
- List.of(1,2,3) vs new ArrayList.
→ List.of — immutable, add/remove → UnsupportedOperationException, null-элементы запрещены. ArrayList — обычный mutable список.

05 Многопоточность и JMM

Для Middle ждут уверенное понимание synchronized, volatile, JMM и пулов потоков.

- Чем процесс отличается от потока?
→ Процесс — изолированная единица ОС со своей памятью. Поток — единица исполнения внутри процесса, память общая.
- Способы создать поток.
→ *extends Thread, implements Runnable/Callable, через ExecutorService, CompletableFuture.*
Предпочтительнее — пулы потоков через Executors.
- Жизненный цикл потока.
→ NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED.
- Как работает synchronized?
→ Захватывает монитор объекта. На методе — монитор this (на static — Class-объект). На блоке — указанного объекта. Только один поток одновременно.
- wait / notify / notifyAll — правила.
→ Только внутри synchronized на том же объекте. wait() освобождает монитор и засыпает. notify будит один, notifyAll — все.
- Почему wait() проверяют в while, а не if?
→ Spurious wakeup — поток может проснуться сам по себе. После пробуждения условие может быть уже невалидным, нужна повторная проверка.
- Зачем volatile?
→ Гарантирует видимость изменений между потоками (запрет кэширования в регистрах/локальном кэше CPU) и запрет reordering. НЕ гарантирует атомарность составных операций (i++).
- Что такое happens-before?
→ Отношение в JMM, гарантирующее видимость. Примеры: synchronized release → acquire, volatile write → read, Thread.start() → действия потока, final-поля после конструктора.
- Atomic-классы — как работают без блокировок?
→ Через CAS (compare-and-swap) — атомарную инструкцию CPU. Lock-free. Классы: AtomicInteger, AtomicLong, AtomicReference.
- Виды пулов в Executors.
→ newFixedThreadPool, newCachedThreadPool, newSingleThreadExecutor, newScheduledThreadPool. В проде часто создают ThreadPoolExecutor вручную, чтобы контролировать очередь и политику отказа.
- Чем опасен Executors.newCachedThreadPool() в проде?
→ Неограниченное количество потоков — при всплеске нагрузки может положить JVM через OutOfMemoryError (unable to create new native thread).
- ThreadLocal — где пригодится и где опасен?
→ Для контекста (например, SecurityContext, транзакционный контекст). Опасен с пулами потоков — значение протекает между задачами, нужен remove() в finally.
- CompletableFuture: thenApply vs thenCompose vs thenCombine.
→ thenApply — преобразование результата. thenCompose — flatMap для CompletableFuture (нужен, когда лямбда возвращает CompletableFuture). thenCombine — объединение двух.
- Как реализовать потокобезопасный singleton?
→ Лучший вариант — enum. Альтернативы: static holder (Initialization-on-demand), double-checked locking с volatile.

- **Deadlock, livelock, starvation.**

→ *Deadlock* — два потока ждут друг друга. *Livelock* — активны, но не прогрессируют (оба уступают). *Starvation* — поток не получает CPU/ресурс.

06 Spring и Spring Boot

В вакансии указан Spring 5–6 и Spring Boot 2–3. Блок обязательный, готовься основательно.

- Что такое IoC и DI?
→ *IoC — принцип: контейнер управляет жизненным циклом объектов. DI — реализация: зависимости внедряются извне (конструктор/сеттер/поле).*
- Какой способ внедрения предпочтительнее?
→ *Constructor injection. Поля можно final, явно видны обязательные зависимости, удобно тестировать без контекста, нет скрытых циклических зависимостей.*
- Жизненный цикл бина.
→ *Инстанцирование → инжект зависимостей → BeanNameAware/BeanFactoryAware/ApplicationAware → @PostConstruct → InitializingBean.afterPropertiesSet → init-method → go home → @PreDestroy → DisposableBean.destroy → destroy-method.*
- Scopes бина.
→ *singleton (default — один на контекст), prototype (новый при каждом запросе), request, session, application, websocket.*
- Чем @Component отличается от @Service, @Repository, @Controller?
→ *Технически почти ничем, все регистрируют бин. Семантически: @Service — бизнес-логика, @Repository — работа с БД + перевод исключений в DataAccessException, @Controller — веб-слой. @RestController = @Controller + @ResponseBody.*
- @Autowired + несколько подходящих бинов — что делать?
→ *@Qualifier("beanName") на точке инжекта, или @Primary на одном из бинов, или имя поля/параметра должно совпадать с именем бина.*
- Как работает @Transactional под капотом?
→ *Spring создаёт прокси (JDK dynamic proxy для интерфейсов или CGLIB для классов). Прокси открывает транзакцию перед методом, коммитит после или откатывает при исключении.*
- Почему @Transactional не работает при self-invocation?
→ *Вызов this.method() идёт минуя прокси. Решение: вынести метод в другой бин или инжектить self через ApplicationContext / @Lazy self.*
- На какие исключения откатывается транзакция по умолчанию?
→ *На RuntimeException и Error. Checked-исключения НЕ откатывают по умолчанию — нужно rollbackFor = Exception.class.*
- Propagation-уровни транзакций.
→ *REQUIRED (default) — присоединяется или создаёт. REQUIRES_NEW — всегда новая, приостанавливает текущую. NESTED — savepoint. SUPPORTS, MANDATORY, NOT_SUPPORTED, NEVER.*
- Что такое Spring Boot?
→ *Spring + автоконфигурация + встроенный сервер (Tomcat/Jetty/Undertow) + стартеры. Минимизирует boilerplate.*
- Что такое стартер?
→ *Готовый набор зависимостей под задачу. Например, spring-boot-starter-web подтянет Spring MVC, встроенный Tomcat, Jackson. Один импорт — и веб-приложение работает.*
- Как работает автоконфигурация?
→ *@EnableAutoConfiguration (входит в @SpringBootApplication) + @Conditional. Список конфигов в META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (в Spring Boot 3+, раньше был spring.factories).*
- Что делает @SpringBootApplication?

→ Это композиция трёх: `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`.

- Как сделать REST-эндпоинт?

→ Класс с `@RestController`, метод с `@GetMapping`/`@PostMapping`. Параметры — `@PathVariable`, `@RequestParam`, `@RequestBody`. Возвращаемый объект сериализуется в JSON через Jackson.

- Как обработать исключение глобально?

→ `@RestControllerAdvice` + `@ExceptionHandler`. Внутри класса — просто `@ExceptionHandler` на методе контроллера.

- `@Async` — что важно знать?

→ Работает через прокси (как `@Transactional`) — не работает при `self-invocation`. Нужен `@EnableAsync`. Default executor может быть неподходящим — часто настраивают свой `TaskExecutor`.

07 Hibernate и Spring Data JPA

В вакансии Hibernate и Spring Data прописаны явно. Ждут уверенную базу.

- Что такое ORM?
→ *Object-Relational Mapping* — маппинг объектов на таблицы БД. Позволяет работать с БД как с объектами, не писать SQL руками.
- Состояния сущности в Hibernate.
→ *Transient* (новый объект, Hibernate не знает), *Persistent/Managed* (связан с сессией), *Detached* (был *persistent*, но сессия закрыта), *Removed* (помечен на удаление).
- Lazy vs Eager загрузка.
→ *Lazy* — связь подгружается только при обращении (прокси). *Eager* — сразу в *SELECT*. Default для *@OneToMany* и *@ManyToMany* — *Lazy*, для *@ManyToOne* и *@OneToOne* — *Eager*.
- Что такое *LazyInitializationException* и как с ним жить?
→ Попытка обратиться к *lazy*-полю после закрытия сессии. Решения: *JOIN FETCH* в запросе, *@EntityGraph*, *DTO-проекции*, *OpenSessionInView* (анти-паттерн в проде).
- Проблема N+1.
→ При итерации по списку на каждую связанную сущность делается отдельный *SELECT*. Решения: *JOIN FETCH*, *@EntityGraph*, *@BatchSize*, *@Fetch(FetchMode.SUBSELECT)*.
- Разница между *get()* и *load()* в Hibernate.
→ *get()* — сразу идёт в БД и возвращает *null* если не найдено. *load()* — возвращает прокси, БД дёргается только при обращении к полям; если нет записи — *ObjectNotFoundException*.
- Разница между *save()*, *persist()*, *merge()*, *update()*.
→ *persist* — для новых (JPA стандарт, не возвращает *id*). *save* — *Hibernate-specific*, возвращает *id*. *merge* — копирует *detached* в *managed*. *update* — прикрепляет *detached*.
- Как Spring Data создаёт реализацию репозитория?
→ Через прокси, генерируемый в рантайме. Имя метода (*findByEmailAndStatus*) парсится и превращается в JPQL.
- Чем отличается *JpaRepository* от *CrudRepository*?
→ *JpaRepository extends PagingAndSortingRepository extends CrudRepository*. Добавляем *flush()*, *saveAllAndFlush()*, работу с *List* вместо *Iterable*, *deleteAllInBatch*.
- *@Query*: JPQL vs native SQL.
→ JPQL работает с сущностями и полями Java. Native SQL — прямой SQL, нужен для специфичных фич БД. Native помечается *nativeQuery = true*.
- Как сделать пагинацию?
→ Метод принимает *Pageable*, возвращает *Page* или *Slice*. Spring Data сам добавит *LIMIT/OFFSET*.
- Оптимистичная блокировка в JPA.
→ Поле *@Version (int/long/timestamp)*. При *update* Hibernate сравнивает версию — если не совпала, бросает *OptimisticLockException*.

08 SQL и PostgreSQL

В отзывах кандидатов Лиги SQL упоминают отдельно — спрашивают понимание, не теорию. Будь готов писать запросы живьём.

- **ACID — расшифровать каждое.**
→ *Atomicity — транзакция целиком или никак. Consistency — БД переходит из валидного состояния в валидное. Isolation — параллельные транзакции не мешают. Durability — после commit данные не теряются.*
- **Уровни изоляции.**
→ *READ_UNCOMMITTED, READ_COMMITTED (default в PostgreSQL), REPEATABLE_READ, SERIALIZABLE. От низкого к высокому: больше консистентности, меньше параллелизма.*
- **Dirty / non-repeatable / phantom read.**
→ *Dirty — чтение незакоммиченных данных. Non-repeatable — два чтения одной строки дают разный результат. Phantom — два SELECT с WHERE возвращают разное число строк.*
- **Типы индексов в PostgreSQL.**
→ *B-tree (default — =, <, >, BETWEEN, ORDER BY), Hash (только =), GIN (массивы, JSONB, full-text), GiST (геометрия), BRIN (большие таблицы с порядком), SP-GiST.*
- **Когда индекс НЕ стоит создавать?**
→ *Маленькие таблицы (full scan быстрее). Колонки с малым числом уникальных значений (boolean, enum с 2–3 значениями). Часто меняющиеся колонки — индекс замедлит INSERT/UPDATE/DELETE.*
- **Виды JOIN.**
→ *INNER — пересечение. LEFT — все из левой + совпадения из правой. RIGHT — наоборот. FULL OUTER — всё. CROSS — декартово произведение.*
- **WHERE vs HAVING.**
→ *WHERE фильтрует строки ДО группировки, работает с индексами. HAVING — ПОСЛЕ группировки, для условий на агрегаты (HAVING SUM(amount) > 1000).*
- **DELETE vs TRUNCATE.**
→ *DELETE — строчное удаление с триггерами и WAL, можно откатить в транзакции. TRUNCATE — быстрая очистка целиком, сбрасывает счётчики автоинкремента.*
- **Как читать EXPLAIN ANALYZE?**
→ *Смотри на Seq Scan (на большой таблице — плохо), Index Scan, Bitmap Heap Scan, типы Join (Nested Loop vs Hash Join), Rows Removed by Filter, разницу estimated vs actual rows.*
- **MVCC в PostgreSQL — как работает?**
→ *Каждая транзакция видит snapshot БД. Вместо изменений — новые версии строк (xmin/xmax). Старые чистит VACUUM.*
- **Оконные функции — что это и зачем?**
→ *ROW_NUMBER(), RANK(), LAG/LEAD, SUM OVER — агрегации поверх результата без сворачивания строк в группы.*

09 Тесты: JUnit, Mockito, TDD

В Лиге тесты — часть инженерной культуры: вакансия прямо требует TDD, JUnit 5, Mockito, Hamcrest, AssertJ.

- Пирамида тестирования.
→ Юнит-тесты (много, быстро) → интеграционные (меньше, медленнее) → e2e (мало, долго).
Чем выше уровень — тем дороже тесты.
- JUnit 5 — основные аннотации.
→ `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`, `@DisplayName`, `@Disabled`, `@ParameterizedTest`, `@Nested`, `@ExtendWith`.
- `@BeforeAll` и `@AfterAll` — что важно?
→ В JUnit 5 они должны быть `static` (или класс аннотирован `@TestInstance(Lifecycle.PER_CLASS)`).
- Mockito: `mock` vs `spy`.
→ `mock()` — пустой объект, все методы возвращают `default` (`null`, `0`, `false`). `spy()` — обёртка над реальным объектом, методы вызываются по-настоящему, если не затоск'ать.
- `when/thenReturn` vs `doReturn/when`.
→ `when(x.method()).thenReturn(y)` — стандартный способ. `doReturn(y).when(x).method()` — нужен для `spy` (иначе вызовется реальный метод) и для `void`-методов.
- `verify` — что это и зачем?
→ Проверка, что метод был вызван (сколько раз, с какими аргументами). `verify(mock, times(2)).method(anyString())`.
- `ArgumentCaptor`.
→ Захват аргумента, с которым был вызван метод, для последующих проверок. Удобно, когда нужно проверить не просто факт вызова, а что именно передали.
- `AssertJ` vs `Hamcrest`.
→ `AssertJ` — *fluent API* (`assertThat(x).isEqualTo(y).hasSize(3)`), более читаемый, богаче. `Hamcrest` — *matchers* (`assertThat(x, equalTo(y))`). В вакансии Лиги оба.
- Что такое TDD?
→ *Red-Green-Refactor*. Сначала пишешь падающий тест (*Red*), потом минимальный код чтобы он прошёл (*Green*), потом рефакторишь без изменения поведения.
- Когда TDD не работает?
→ Исследовательский код (прототипы), UI-разработка, GUI, задачи где требования постоянно меняются в процессе.
- Тестировать ли `private` методы?
→ Нет — тестируются через публичные. Если `private` стало сложно — это сигнал, что надо выделить отдельный класс.
- Что такое `Testcontainers`?
→ Библиотека для запуска реальных БД/брокеров в `Docker` из тестов. Нужна для интеграционных тестов, которые проверяют реальную работу с `PostgreSQL/Kafka`, а не моки.

10 Docker, Jenkins, Linux

► Docker

- Образ (image) vs контейнер (container).
→ Образ — неизменяемый шаблон (как класс). Контейнер — запущенный экземпляр образа (как объект). Из одного образа — много контейнеров.
- Dockerfile: основные инструкции.
→ FROM (базовый образ), COPY/ADD, RUN, WORKDIR, EXPOSE, ENV, CMD/ENTRYPOINT, VOLUME, USER.
- CMD vs ENTRYPOINT.
→ ENTRYPOINT — основная исполняемая команда, сложно переопределить. CMD — аргументы по умолчанию, легко переопределить в docker run.
- Multi-stage build — зачем?
→ Собираешь в одном образе (с Maven/Gradle), а в финальный копируешь только JAR. Финальный образ маленький и без build-инструментов.
- docker-compose.
→ Инструмент для запуска нескольких связанных контейнеров одной командой через yaml. Типовой сценарий: приложение + БД + Redis + Kafka.

► Jenkins и CI/CD

- Что такое CI и CD?
→ CI — автоматическая сборка/тесты/статический анализ при каждом push. CD — Continuous Delivery (готовность к деплою) или Continuous Deployment (автодеплой в прод).
- Jenkins Pipeline — что это?
→ Декларативный или scripted pipeline, описан в Jenkinsfile в корне репозитория. Этапы (stages): checkout, build, test, deploy.
- Declarative vs Scripted Pipeline.
→ Declarative — более структурированный, с фиксированным синтаксисом (pipeline { agent, stages }). Scripted — Groovy-код, более гибкий.

► Linux — базовый минимум

- Как посмотреть логи запущенного Java-процесса?
→ tail -f /var/log/app.log, или journalctl -u service-name -f, или через docker logs -f container.
- Как найти процесс, занявший порт?
→ lsof -i :8080 или ss -tulpn | grep 8080 или netstat -tulpn | grep 8080.
- Как отправить сигнал процессу?
→ kill PID (SIGTERM, мягкое завершение), kill -9 PID (SIGKILL, жёсткое). Команда pkill/killall — по имени.
- Чем отличаются soft link и hard link?
→ Soft (ln -s) — указатель на путь, работает через каталоги. Hard — второе имя для того же inode, нельзя через каталоги и на другую файловую систему.

11 Практические задачи (live-coding)

По отзывам кандидатов — в Лиге на Java Middle любят живое кодирование с рассуждением. Задачи прикладные, без LeetCode Hard.

► Задача 1. Найти дубликаты в списке

Формулировка: дан `List<Integer>`. Вернуть `Set<Integer>` чисел, которые встречаются более одного раза.

```
public Set<Integer> findDuplicates(List<Integer> list) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> duplicates = new HashSet<>();
    for (Integer n : list) {
        if (!seen.add(n)) {
            duplicates.add(n);
        }
    }
    return duplicates;
}
```

Или через Stream API:

```
public Set<Integer> findDuplicates(List<Integer> list) {
    return list.stream()
        .collect(Collectors.groupingBy(n -> n, Collectors.counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .collect(Collectors.toSet());
}
```

Что спросят: сложность (первый вариант $O(n)$, второй — тоже $O(n)$, но с бóльшей константой), можно ли без `HashSet`, что будет если в списке `null`.

► Задача 2. Группировать сотрудников по отделам

Формулировка: дан `List<Employee>`, у каждого есть имя и отдел. Вернуть `Map<String, List<Employee>>` — отдел → список сотрудников.

```
public Map<String, List<Employee>> groupByDepartment(List<Employee> employees) {
    return employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment));
}
```

Частое продолжение: «А теперь получить `Map<String, Long>` — отдел → количество сотрудников». Ответ — `Collectors.counting()` как downstream:

```
Map<String, Long> counts = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()
    ));
```


► Задача 3. SQL-запрос вживую

Такого типа задачу в Лиге дают почти всегда. Умеешь ли ты писать JOIN, GROUP BY, HAVING и объяснять план.

Типовая формулировка: есть таблицы employees(id, name, department_id, salary) и departments(id, name). Выведи названия отделов, где средняя зарплата выше 100 000, отсортируй по убыванию средней.

```
SELECT d.name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON d.id = e.department_id
GROUP BY d.name
HAVING AVG(e.salary) > 100000
ORDER BY avg_salary DESC;
```

Почти наверняка продолжат спрашивать

- Какие индексы добавил бы? — на employees.department_id, и возможно покрывающий (department_id, salary).
- Почему AVG в HAVING, а не в WHERE? — WHERE не видит агрегатов, они считаются после группировки.
- Как найти сотрудников с зарплатой выше средней по ИХ отделу? — через оконную функцию AVG() OVER (PARTITION BY department_id).

► Задача 4. REST-контроллер на Spring Boot

Формулировка: «Напиши контроллер, который принимает POST /api/users с телом JSON, валидирует его и сохраняет через сервис». Типичное задание вживую.

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public UserDto create(@RequestBody @Valid CreateUserRequest request) {
        return userService.create(request);
    }

    @GetMapping("/{id}")
    public UserDto getById(@PathVariable Long id) {
        return userService.findById(id);
    }
}

public record CreateUserRequest(
    @NotBlank String name,
    @Email String email,
    @Min(18) int age
) {}
```

На что обратят внимание

- Constructor injection, а не @Autowired над полем.
- DTO, а не Entity — иначе утечка внутренних полей наружу.
- @Valid для запуска валидации Bean Validation.
- Правильные HTTP-статусы: 201 Created для POST, 200 OK для GET, 204 No Content для DELETE.
- Как обработать MethodArgumentNotValidException глобально — через @RestControllerAdvice.

► Задача 5. Написать unit-тест к сервису

Формулировка: «Вот сервис UserService, зависит от UserRepository и EmailSender. Напиши тест для метода register(email, name), который создаёт пользователя, сохраняет и отправляет приветственное письмо». С учётом TDD в Лиге задают часто.

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private EmailSender emailSender;

    @InjectMocks
    private UserService userService;

    @Test
    void shouldRegisterUserAndSendEmail() {
        // given
        String email = "alice@example.com";
        String name = "Alice";
        User saved = new User(1L, name, email);
        when(userRepository.save(any(User.class))).thenReturn(saved);

        // when
        User result = userService.register(email, name);

        // then
        assertThat(result.getId()).isEqualTo(1L);
        assertThat(result.getEmail()).isEqualTo(email);

        ArgumentCaptor<User> captor = ArgumentCaptor.forClass(User.class);
        verify(userRepository).save(captor.capture());
        assertThat(captor.getValue().getName()).isEqualTo(name);

        verify(emailSender).sendWelcome(email);
    }
}
```

Что важно

- Схема given-when-then — читаемо и принято в индустрии.
- ArgumentCaptor, чтобы проверить, что именно передали в save().
- AssertJ (assertThat(...)) — чище, чем assertEquals. В Лиге его любят.
- Проверка вызова emailSender — не забыть про побочные эффекты.

► Задача 6. N+1 — найти и починить

Формулировка: «Вот код репозитория и сервиса. Приложение тормозит на выводе списка заказов с их товарами. В чём проблема, как исправить?»

```
@Entity
public class Order {
    @Id
    private Long id;

    @OneToMany(mappedBy = "order") // LAZY by default
    private List<OrderItem> items;
}

// В сервисе:
List<Order> orders = orderRepository.findAll();
for (Order o : orders) {
    System.out.println(o.getItems().size()); // +1 запрос на каждый Order
}
```

Что случилось

1 запрос на `findAll()` + N запросов на каждую коллекцию `items` — та самая N+1. При 1000 заказов — 1001 SQL-запрос.

Как чинить

- JOIN FETCH в запросе: `@Query("select o from Order o left join fetch o.items")`.
- `@EntityGraph(attributePaths = "items")` на методе репозитория.
- `@BatchSize(size = 100)` на коллекции — Hibernate подгрузит `items` пачками.
- DTO-проекция через JPQL — сразу плоский объект без сущности.

ЛОВУШКА • Ловушка

«Просто поставить EAGER» — неправильный ответ. EAGER грузит коллекцию ВСЕГДА, даже когда она не нужна. Это медленнее и съедает память. Правильно — LAZY по умолчанию + FETCH там, где надо.

► Задача 7. Потокобезопасный кэш

Формулировка: «Напиши простой кэш `Map<String, String>`, в который пишут и читают из разных потоков. Чтобы работало правильно».

Минимальный правильный вариант

```
public class SimpleCache {
    private final Map<String, String> cache = new ConcurrentHashMap<>();

    public String get(String key, Function<String, String> loader) {
        return cache.computeIfAbsent(key, loader);
    }
}
```

Что спросят

- Почему `ConcurrentHashMap`, а не `HashMap`? — `HashMap` в многопоточке может породить бесконечный цикл (Java 7) или потерять данные.
- Почему `computeIfAbsent`, а не `containsKey` + `get` + `put`? — гонка между проверкой и вставкой. `computeIfAbsent` атомарный.

- Что если loader долго работает? — `computeIfAbsent` блокирует bucket, другие потоки с тем же ключом будут ждать. Для долгих операций лучше `CompletableFuture` или `Caffeine`.
- Как ограничить размер? — не стандартной `ConcurrentHashMap`, нужен `Caffeine` / `Guava Cache` с `maxSize` и `TTL`.

► Задача 8. Анаграммы

Классика для разминки. Формулировка: «Две строки — анаграммы, если состоят из одних и тех же букв в разном порядке. Проверь».

```
public boolean isAnagram(String a, String b) {  
    if (a == null || b == null || a.length() != b.length()) return false;  
    char[] aChars = a.toLowerCase().toCharArray();  
    char[] bChars = b.toLowerCase().toCharArray();  
    Arrays.sort(aChars);  
    Arrays.sort(bChars);  
    return Arrays.equals(aChars, bChars);  
}
```

Сложность: $O(n \log n)$ из-за `sort`. Можно ли быстрее — да, за $O(n)$ через массив счётчиков `int[26]` (для ASCII) или `HashMap<Character, Integer>` для Unicode.

12 План подготовки + чек-лист

► За 2–3 недели

1. Прорешать 20–30 задач на LeetCode (Easy + Medium) на массивы, строки, хэш-таблицы, Stream API.
2. Поднять pet-проект на Spring Boot 3 + PostgreSQL + Testcontainers. Если есть — отрефакторить.
3. Прокатать Hibernate: разобраться с N+1, состояниями сущности, JOIN FETCH, @EntityGraph.
4. Повторить SQL: JOIN, GROUP BY, HAVING, оконные функции, EXPLAIN ANALYZE.
5. Написать 5–10 unit-тестов с Mockito + AssertJ. Попробовать подход TDD.

► За неделю

6. 2–3 мок-интервью: проси друзей или используй pramp.com / interviewing.io.
7. Прорешать вопросы из этого гайда вслух — проговорить реально важно.
8. Подготовить 2–3 истории по схеме: проблема → что сделал → результат (с цифрами).
9. Освежить Docker: написать Dockerfile и docker-compose для pet-проекта.

► В день собеса

- Проверить камеру, микрофон, интернет.
- IDE наготове (IntelliJ IDEA), чтобы не тратить время на настройку.
- Лист бумаги — для схем и набросков.
- Рассуждать вслух. Молчание для интервьюера хуже, чем «дай минуту подумать».
- Не знаешь — скажи честно: «не сталкивался, но предположу, что...». Честность ценят.
- В конце задай 2–3 вопроса: про проект, команду, процесс code review.

ВНИМАНИЕ · Про банковские проекты

Большинство Java-вакансий в Лиге — в банки (по актуальной вакансии — антифрод крупного банка). Будь готов, что проверку СБ и оформление по ТК РФ делают серьёзно. Удалёнка возможна, но обычно из РФ.

► Финальный чек-лист

Блок	Готов, если можешь...
Java Core	Объяснить контракт equals/hashCode на примере и показать, как его нарушение ломает HashSet
Коллекции	Рассказать, как устроен HashMap в Java 8+, и почему плохо держать mutable-объект ключом
Многопоточность	Написать потокобезопасный singleton и объяснить роль volatile в DCL
JMM	Объяснить happens-before на примере volatile и synchronized
Spring	Рассказать, почему @Transactional не работает при self-invocation

Hibernate	Найти N+1 в коде и предложить 2+ способа починить
SQL	Написать запрос с JOIN + GROUP BY + HAVING и обосновать индекс
Тесты	Написать тест с Mockito + ArgumentCaptor + AssertJ
Docker	Написать Dockerfile с multi-stage build для Spring Boot
Live-coding	За 15 минут написать чистое решение задачи типа 'group by department'

Удачи на собеседе!

`// git commit -m "offer signed"`

JavaJub